

# Odoo Technical Orientation

01

## Class 01 · Foundation Track

Understand how Odoo is built from the ground up — its three-tier architecture, its module system, and the development environment you will work in all course long.

---

### FORMAT

**30 Hours Total**

Beginner Level · Hands-on

### STACK

**Odoo 17 – 19**

Python 3.10+ · PostgreSQL 15

### YOUR TRAINER

**Zahed Hasan** · Odoo Technical Trainer

[thezhasan.com](https://thezhasan.com) · [linkedin.com/in/zahedhasan](https://linkedin.com/in/zahedhasan)

# What is Odoo?

An open-source, modular ERP built on Python and PostgreSQL. Odoo ships a unified suite of business applications that share a single database — sales, accounting, inventory and HR all work on the same records.

|                          |                                                                                      |
|--------------------------|--------------------------------------------------------------------------------------|
| <b>Open source</b>       | Community Edition is free on GitHub — you will read and extend real production code. |
| <b>Modular</b>           | Install only what the business needs; every app is just another module.              |
| <b>Python + XML + JS</b> | Python backend, XML view definitions, JavaScript frontend — one coherent stack.      |
| <b>Customizable</b>      | Extend anything through inheritance instead of modifying core files.                 |

- **Community Edition** — free and open source ([github.com/odoo/odoo](https://github.com/odoo/odoo))
- **Enterprise Edition** — paid; adds extra features, support and mobile apps

## ODOO 17-19 BY THE NUMBERS

17

Latest stable version — this course covers 17, 18 and 19

11M+

Users worldwide, from startups to enterprises

30+

Official modules maintained by Odoo S.A.

44K+

Community apps on the Odoo Apps store

Both editions share the same core and the same architecture — everything in this course applies to either one.

# Install only what the business needs

Every app below is a module with the same structure you will learn to build — and each one plugs into the same shared database.

|                 |               |           |            |
|-----------------|---------------|-----------|------------|
| CRM             | Sales         | Purchase  | Inventory  |
| Accounting      | Manufacturing | HR        | Payroll    |
| Project         | Helpdesk      | eCommerce | Website    |
| Email Marketing | POS           | Fleet     | Events     |
| Survey          | Blog          | Forum     | Live Chat  |
| Appointments    | Knowledge     | Sign      | + more ... |



In this course you will build a module that sits right next to these — same folder structure, same manifest, same rules. By Class 17, yours will be deployed on a real server.

# Three tiers — each only talks to its neighbour

01

## PRESENTATION

The web client — a Single Page Application in the browser

OWL · QWeb · XML views · JavaScript



JSON-RPC over HTTP(S)

02

## LOGIC / APPLICATION

The Python server — business rules, security, data flow

Odoo ORM · Controllers · Access control · Module system



SQL generated by the ORM

03

## DATA

PostgreSQL database + filestore on disk

Tables · jsonb · Sequences · Attachments

### THE ONE RULE

Each tier communicates only with the layer next to it. The browser never speaks to PostgreSQL directly — every request goes through the Python server.

### WHERE YOUR CODE LIVES

Almost entirely in Tier 02 — models in Python, views in XML, the occasional controller. Tiers 01 and 03 are handled by the framework, which is why Odoo development moves fast.

### REMEMBER

**Browser → Python → PostgreSQL**, and back. Keep this picture in your head — every debugging session starts here.

# The browser renders views — you only declare them

Odoo's frontend is a Single Page Application. It exchanges data with the server over XML-RPC / JSON-RPC and renders the whole interface in the browser — no page reloads.

HTML5 · CSS3 · JavaScript · OWL · QWeb

## OWL Framework

Odoo Web Library — a reactive component framework, similar in spirit to React or Vue. Powers all modern UI since Odoo 14.

## QWeb Templates

XML-based templating for server-side PDF rendering and client-side components. Directives like t-if, t-foreach, t-call.

## XML View Definitions

Form, List, Kanban, Calendar and Search views written as XML. The server converts them to JSON; OWL renders the result.

## HOW A VIEW REQUEST FLOWS

- 1 User opens Sales → Orders in the web client
- 2 OWL's ActionManager fires a JSON-RPC call to the server
- 3 The server returns the view definition plus the record data
- 4 OWL renders the List view component with that data
- 5 The user sees the table of records — no page reload

As a developer you rarely write raw HTML. You define **XML view files** — Odoo handles rendering. Views are stored in the database, so they can be extended without touching the original files.

# One Python server does all the thinking

Requests, business rules, permissions and data flow are all processed here — each request handled synchronously by a worker.

| COMPONENT        | WHAT IT DOES                                                                      | KEY FEATURES                                        |
|------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------|
| Python Server    | Werkzeug WSGI app. Each request is processed synchronously, one worker at a time. | HTTP · multi-worker · long-polling (bus)            |
| Odoo ORM         | Maps Python classes to PostgreSQL tables — no raw SQL needed for most operations. | search() · create() · write() · @api.depends        |
| Access Control   | Multi-level security. Every record access is checked against the user's rights.   | ir.model.access.csv · res.groups · ir.rule · sudo() |
| Module System    | Functionality is split into addons with declared dependencies.                    | __manifest__.py · --addons-path · -u for upgrades   |
| HTTP Controllers | Custom web routes for REST endpoints, webhooks and website pages.                 | @http.route() · type=json/http · auth=public/user   |

## WORKFLOWS & STATES

Odoo uses **state fields and plain Python methods** instead of a formal workflow engine — a button click triggers a server method that updates the record's state. Scheduled jobs run through `ir.cron`, and the chatter (`message_post`) keeps an audit trail on every record.

# PostgreSQL only — and files live outside the database

## PostgreSQL

- › The only supported database — no MySQL, no SQLite
- › Uses PostgreSQL-specific features: `jsonb` columns, sequences, advisory locks
- › Minimum version 12 — version 15+ recommended
- › One database per company; schema auto-managed by the ORM — you never write

### CREATE TABLE

## Filestore (attachments)

Binary fields — attachments, images — are stored as files on disk, not in the database.

```
~/local/share/odoo/filestore/
├─ {db_name}/
│   └─ a1/b2/c3... ← SHA-1 hash dirs
# odoo.conf
data_dir = /var/lib/odoo
```

## HOW THE ORM NAMES THINGS

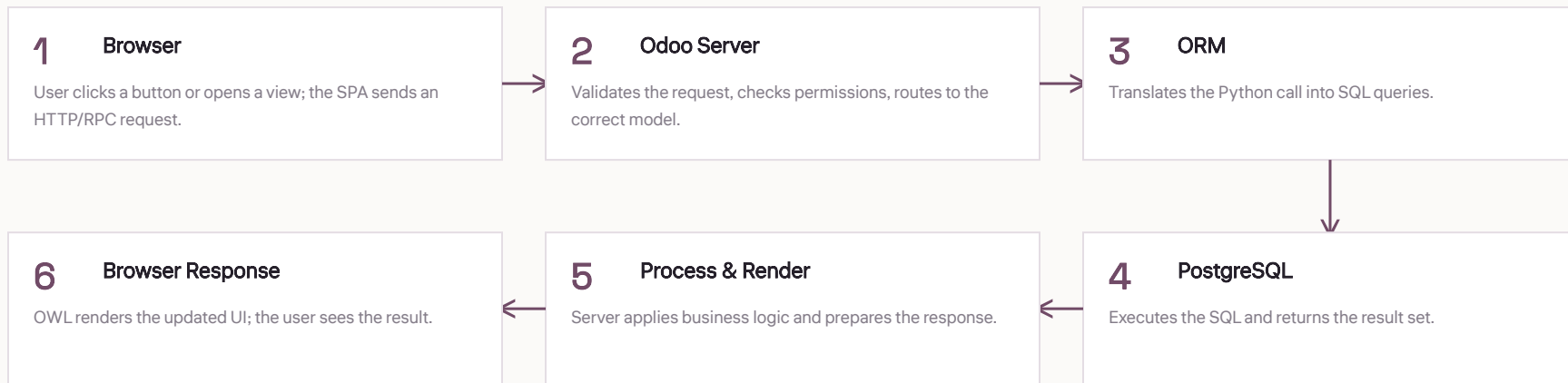
|                                |                                |
|--------------------------------|--------------------------------|
| <code>sale.order</code>        | <code>sale_order</code>        |
| <code>res.partner</code>       | <code>res_partner</code>       |
| <code>account.move</code>      | <code>account_move</code>      |
| <code>fields.Char()</code>     | <code>VARCHAR column</code>    |
| <code>fields.Many2one()</code> | <code>INTEGER FK column</code> |

Dots become underscores; field types become column types — automatically.

When you create a database from Odoo's database manager, the ORM builds the entire schema itself. Your Python model **is** the table definition — keep them in sync mentally and debugging gets much easier.

**Backups:** the PostgreSQL database and the filestore must be backed up — and restored — **together**. Restoring only one of them breaks every attachment.

# One click, six hops through the stack



## ALL CLIENT-SERVER TRAFFIC IS JSON-RPC

```
POST /web/dataset/call_kw
{
  "model": "sale.order",
  "method": "action_confirm",
  "args": [[42]], "kwargs": {}
}
```

## FRONTEND ASSETS

Custom JavaScript modules drive behaviour; SCSS handles styling. Odoo bundles everything through its own asset system (`ir.assets`) — you declare assets in the manifest, Odoo serves the bundles.

# MVC in Odoo — models and views do 90% of the work

## M · Model

models/\*.py

Python classes extending models.Model. Fields, business logic, computed values, constraints.

```
from odoo import models, fields

class ProjectTask(models.Model):
    _name = 'project.task'
    _description = 'Task'

    name = fields.Char(required=True)
    state = fields.Selection([
        ('draft', 'Draft'),
        ('done', 'Done'),
    ])

```

## V · View

views/\*.xml

XML files defining which fields appear, in what layout. Stored as ir.ui.view records.

```
<record id="view_task_form"
        model="ir.ui.view">
    <field name="name">Task Form</field>
    <field name="model">project.task</field>
    <field name="arch" type="xml">
        <form>
            <field name="name"/>
            <field name="state"/>
        </form>
    </field>
</record>

```

## C · Controller

controllers/\*.py

Python classes handling HTTP routes — website pages, REST APIs, custom webhooks.

```
from odoo import http
from odoo.http import request

class TaskController(http.Controller):

    @http.route('/api/tasks',
                type='json', auth='user')
    def get_tasks(self):
        tasks = request.env['project.task']
        return tasks.search_read(
            [], ['name'])

```

In practice, the built-in RPC controller handles most operations — you will write controllers only for **external APIs and website pages**.

# Everything is a module — including the core

A module (addon) is a directory of Python code, XML views, CSV data and static assets. Installing it registers the models, loads the views and seeds the data.



## Models

Python classes. On install, the ORM creates the database tables for you.



## Views

XML files, loaded into `ir.ui.view` records on install.



## Data & demo

CSV / XML records seeded into the database on install.



## Security

`ir.model.access.csv` grants CRUD rights per user group.

## MODULE LIFECYCLE

### Install

Creates tables, loads views, seeds data. Dependencies are installed first.

### Upgrade

Updates the schema, reloads views, re-applies data. Run with the `-u` flag.

### Uninstall

Removes views and data, drops the tables. Use with caution.

```
$ python odoo-bin -u my_module -d mydb
```

Upgrade one module in one database — your most-typed command this course.

## DEPENDENCIES & DISCOVERY

Modules declare dependencies in `__manifest__.py` and install in dependency order.

Odoo discovers modules via `--addons-path` (comma-separated) — keep your custom modules in a directory separate from the Odoo source.

# One layout for every module — learn it once

```

my_module/
├── __manifest__.py    ← metadata & config
├── __init__.py       ← package init
├── models/
│   ├── __init__.py   ← imports
│   └── my_model.py    ← logic
├── views/
│   ├── my_views.xml  ← UI definitions
│   └── menu.xml       ← menus
├── security/
│   ├── ir.model.access.csv
│   └── security.xml   ← record rules
├── data/              ← seeded records
├── report/            ← PDF reports (QWeb)
└── controllers/      ← HTTP routes

```

## MINIMUM VIABLE MODULE — 3 FILES

|                                 |                                       |
|---------------------------------|---------------------------------------|
| <code>__manifest__.py</code>    | Module metadata & configuration       |
| <code>__init__.py</code>        | Makes the directory a Python package  |
| <code>models/__init__.py</code> | Only if you add models — imports them |

## \_\_manifest\_\_.py TEMPLATE

```

{
    'name': 'My Module',
    'version': '17.0.1.0.0',
    'category': 'Custom',
    'author': 'SDTEKH',
    'depends': ['base'],
    'data': [
        'security/ir.model.access.csv',
        'views/my_views.xml',
    ],
    'installable': True,
    'application': True,
}

```

Every directory and file has exactly one responsibility — and this structure is **identical across all Odoo modules**. Learn it once, apply it everywhere.

# Never modify the core — inherit and extend

## Model Inheritance

`_inherit`

Add fields, override methods, change behaviour on existing models.

```
class SaleOrder(models.Model):
    _inherit = 'sale.order'

    # add a custom field
    approval_notes = fields.Text(
        string='Approval Notes')

    # override a method
    def action_confirm(self):
        self._check_approval()
        return super().action_confirm()
```

## View Inheritance

`inherit_id + XPath`

Add, replace or remove elements in existing views using XPath selectors.

```
<record id="sale_form_inherit"
        model="ir.ui.view">
    <field name="inherit_id"
          ref="sale.view_order_form" />
    <field name="arch" type="xml">
        <xpath expr="//field[@name='note']"
              position="before">
            <field name="approval_notes" />
        </xpath>
    </field>
</record>
```

## Data Extension

`new records + overrides`

Menus, actions, sequences, templates — without touching existing records.

```
<menuitem id="menu_custom"
          name="Custom Tasks"
          parent="project.menu_pm"
          action="action_task" />

<record id="base.action_users"
        model="ir.actions.act_window">
    <field name="domain">
        [('active', '=', True)]
    </field>
</record>
```

**The golden rule:** your custom modules live in their own addons folder. When Odoo upgrades, the core stays untouched — you only update your own module. That is what makes Odoo maintainable at scale.

## BEFORE WE START

# Set the machine up once — correctly

## RUNTIME

|                |                         |          |
|----------------|-------------------------|----------|
| Python 3.10+   | 3.12 may have issues    | Required |
| pip + venv     | virtual env recommended | Required |
| PostgreSQL 14+ | psql must be in PATH    | Required |
| wkhtmltopdf    | for PDF reports         | Required |
| Node.js 16+    | for less/sass compile   | Required |

## VSCODE EXTENSIONS

|                    |                              |
|--------------------|------------------------------|
| Python (ms-python) | IntelliSense + debugging     |
| Pylance            | Type checking, autocomplete  |
| XML (redhat)       | XML validation, autocomplete |
| Odoo Snippets      | Snippets for views           |

## TOOLS

|                   |                          |          |
|-------------------|--------------------------|----------|
| VS Code           | best IDE for Odoo dev    | Required |
| Git               | clone the Odoo source    | Required |
| pgAdmin / DBeaver | DB browser for debugging | Optional |
| Docker            | containerized setup      | Optional |

## SYSTEM REQUIREMENTS

RAM 4 GB (8 GB recommended) · 10 GB free storage · Ubuntu 22.04, macOS or Windows with WSL2

## QUICK INSTALL CHECK

```
$ python3 --version
$ psql --version
$ node --version
$ git --version
```

# 17 classes, 8 tracks — from setup to deployment

17 Classes · 30 Hours · Odoo 17, 18 & 19

## 01 · Foundation

CLASS 01-02

Architecture, setup, dev environment, module structure

## 02 · ORM & Data

CLASS 03-05

Models, fields, relations, computed fields, constraints

## 03 · Views & UI

CLASS 06-08

Form, List and Kanban views, QWeb, OWL components

## 04 · Backend Logic

CLASS 09-11

Controllers, workflows, security, automated actions

## 05 · Advanced

CLASS 12-13

Multi-company, performance, debugging, reporting

## 06 · FastAPI

CLASS 14-15

FastAPI integration, external APIs, webhooks

## 07 · Deployment

CLASS 16

Linux server, Nginx, SSL, CI/CD pipeline

## 08 · Capstone

CLASS 17

A complete custom module, built and deployed

## CAPSTONE PROJECT

**Service Request Management System** — a full custom module with models, views, workflows, security rules, PDF reports and FastAPI integration, deployed to a VPS.

**By the end of the course:** build production-ready modules from scratch · own the full development lifecycle · deploy and maintain Odoo applications · integrate external APIs with FastAPI

# Questions?

Let's build something great with Odoo.

---

## YOUR TRAINER

**Zahed Hasan · SDTEKH**

[zahed.hasan.rabbi@gmail.com](mailto:zahed.hasan.rabbi@gmail.com)

[thezhasan.com](http://thezhasan.com) · [linkedin.com/in/zahedhasan](https://linkedin.com/in/zahedhasan)

## COMING UP NEXT

**02**

### **Creating Your First Custom Module**

Scaffold, manifest, install — your first addon end to end

---

**03**

### **Models & Fields**

Field types, relations and the ORM in practice

---

**04**

### **Computed Fields & Constraints**

Logic that stays in sync, automatically

---